



TP03 — SISTEMAS DISTRIBUÍDOS

Replicação e Tolerância a Falhas

Cluster Store **primário-backup** — o recurso R replicado em 3 nós, sobre o Cluster Sync do TP2

Continuação do TP2: a seção crítica agora escreve de verdade em um cluster replicado, tolerante a quedas e omissões — rodando em Kubernetes e na nuvem AWS.

Fábio Tavares

Do TP2 ao TP3: o recurso R agora é real



TP2 — Exclusão Mútua

- O Cluster Sync coordena quem acessa R, sem nó mestre
- Multicast ordenado (RabbitMQ R_topic) + fila F privada
- Na seção crítica, um sleep de 0,2–1s SIMULAVA o acesso
- Foco: coordenar o acesso de forma justa



TP3 — Replicação + Falhas

- A seção crítica **ESCREVE DE VERDADE** no recurso R
- Novo Cluster Store: 3 réplicas em primário-backup
- Tolerar falha por queda e por omissão de nós
- Foco: replicar o dado e sobreviver a falhas

O PROBLEMA PROPOSTO

Replicar o recurso R e sobreviver a falhas



Sem middleware na parte nova: toda a comunicação do Cluster Store (escrita, replicação, PING, snapshot) é HTTP puro, feito à mão.



3 réplicas do Store

O recurso R vive em 3 nós. Qualquer réplica aceita a escrita — se não for a primária, repassa ao primário.



Escolha aleatória

Cada requisição sorteia o recurso (R1–R5) e cada acesso sorteia a réplica do Store — ponto extra da proposta.



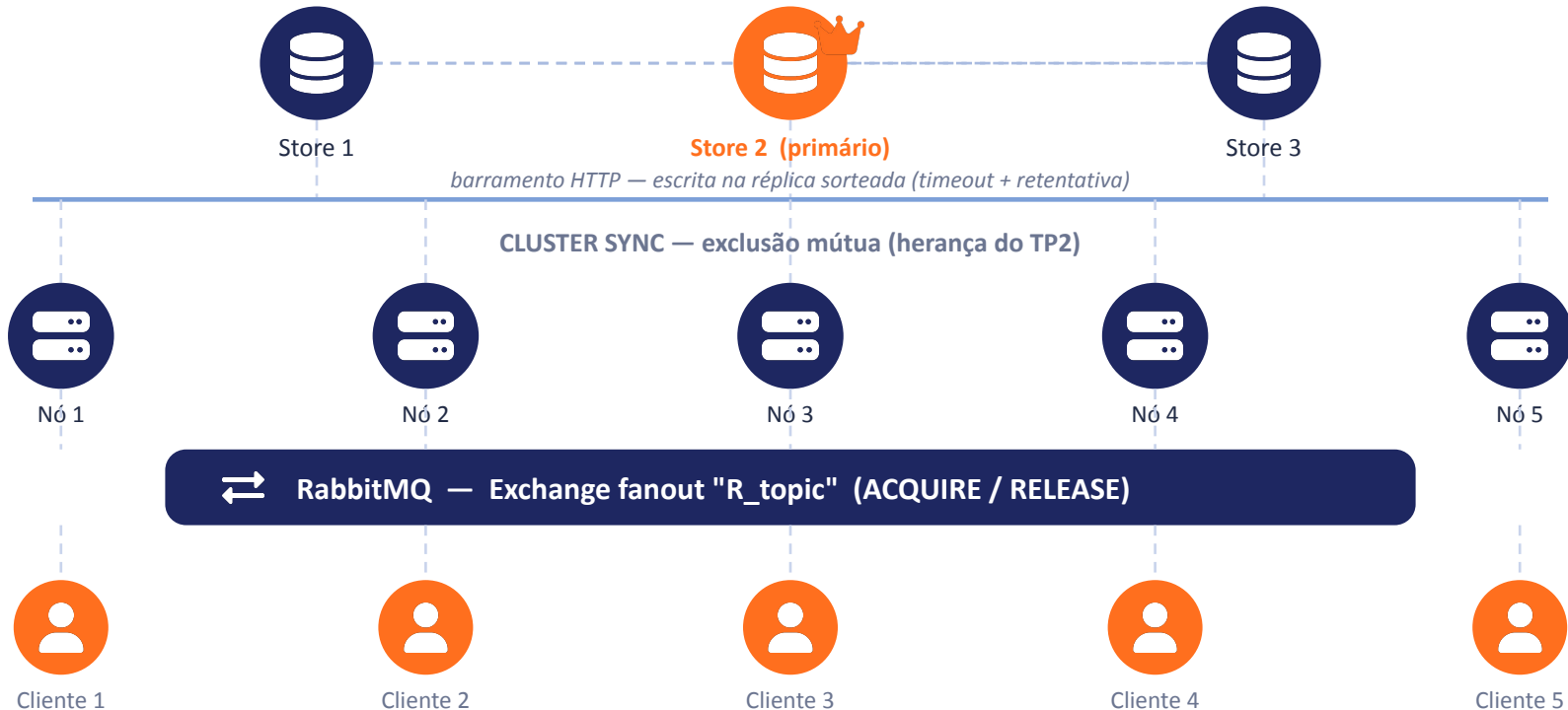
Queda e omissão

PINGs periódicos + timeout detectam nós mortos. O sistema continua servindo e se recupera sozinho.

VISÃO GERAL DA SOLUÇÃO

Arquitetura: dois clusters e o broker

CLUSTER STORE — 3 réplicas · primário-backup · HTTP puro



Como uma escrita é replicada

W1

Sync escreve

Na seção crítica, sorteia uma réplica do Store e envia a escrita via HTTP.

W2

Repassa ao primário

Se a réplica sorteada não é a primária, ela encaminha o pedido ao primário.

W3

Primário replica

O primário aplica localmente e envia a atualização a todos os backups vivos.

W4

Backups confirmam

Cada backup aplica a escrita e devolve um ACK ao primário.

W5

Commit

Com os ACKs, o primário responde COMMITTED (com seq e valor novo) ao Sync.



Resposta ao cliente

O Sync publica RELEASE e devolve ao cliente o valor efetivamente gravado.

O que acontece quando um nó do Store cai



CASO 2.1

Backup cai

Os outros detectam por timeout de PING. As escritas seguem sem ele; ao voltar, re-sincroniza o estado por snapshot.



CASO 2.2

Falha com pedido em andamento

O Sync toma timeout e retenta em outra réplica sorteada. A deduplicação por req_id impede escrita dupla.



CASO 2.3

Primário cai

Eleição por época: o menor ID vivo assume. A escrita é retentada no novo primário; o antigo volta como backup.

Rodando em Kubernetes, na nuvem AWS



Docker Compose

15 containers (RabbitMQ, 3 Store, 5 Sync, 5 Cliente, Dashboard) sobem com um comando.



Kubernetes

StatefulSets dão identidade estável e DNS por pod. kubectl delete pod → o cluster se cura sozinho.



AWS EC2 (k3s)

Deploy na nuvem por script. Dashboard público na porta 30500 — a turma acompanha pelo navegador.



Dashboard ao vivo: a topologia animada mostra cada mensagem do protocolo, a eleição do primário e a consistência das 3 réplicas em tempo real.



CONCLUSÃO

Replicação real, tolerante a falhas e na nuvem



Replicação primário-backup

O recurso R vive em 3 réplicas consistentes; qualquer uma aceita a escrita e o primário propaga aos backups.



Tolerância a falhas comprovada

Queda e omissão são detectadas por PING; eleição, retentativa e re-sincronização mantêm o serviço no ar.



Kubernetes + AWS

Todo o sistema roda em containers, com self-healing no Kubernetes e implantação pública na EC2 da AWS.